GUIA DE ESTUDO

Instâncias do PAPT: construção, tempo e GAP

Roteiro prático para organizar dados, validar a formulação e conduzir experimentos com Python, Julia, JuMP e CPLEX.

Objetivo. Construir uma rotina experimental reproduzível: partir de uma instância pequena de referência, criar cenários sintéticos controlados, executar o solver e registrar tempos, bound, GAP e métricas da solução.

Caminho recomendado

1. Validar o JSON canônico referencia_v001.json.
2. Confirmar os indicadores esperados com Julia/CPLEX.
3. Gerar instâncias sintéticas mudando um fator por vez.
4. Executar repetições com limites e threads registrados.
5. Consolidar o CSV e interpretar status, tempo, bound e GAP.

Este guia acompanha a hierarquia consolidada em modelagem_pl/. A pasta de execução do modelo é modelo/projeto_ic/model_ic/; mantenha originais e dados de diagnóstico fora da pasta automática instancias/.

1. Instância de referência e valores de controle

A instância canônica é modelo/projeto_ic/model_ic/instancias/referencia_v001.json. Ela contém 3 professores, 4 turmas e 2 disciplinas. A versão que chegou originalmente foi preservada em modelo/referencias/instancia_de_referencia_recebida.json para manter rastreabilidade.

Correções de consistência aplicadas ao arquivo canônico:

- Remoção da repetição de `Qua-vesp` em `turnos_semanais`.
- Inclusão de `Ter-vesp`, usado por `TEC217-TP02`.
- Normalização de `9:30` para `09:30`.
- Correção da disciplina de `TEC217-TP01` para `TEC217`.

Propriedade	Valor
Professores	3
Turmas / disciplinas	4 / 2
Slots distintos (dia + início)	8
Turnos semanais distintos	6
Pares professor-turma	12
Pares elegíveis	6 / 12 (50%)

Sob a função objetivo ponderada atual, enumeração independente em Python encontrou 3 atribuições viáveis. A melhor atribuição fornece os seguintes valores:

F	s_min	C	g_max	Z
17	3	8	1	9,9

Não confundir: estes valores são referência matemática independente; os tempos, o bound e o GAP ainda precisam ser medidos em Julia/CPLEX. Se o solver retornar valores diferentes, confira os parâmetros, as restrições e a função objetivo antes de avançar.

2. Construção e validação do JSON

2.1 Oferta real: PDF → JSON intermediário

```
python -m pip install -r ferramentas/requirements.txt
python ferramentas/extrator_oferta.py "dados/origem/oferta_semestre_2026_2.pdf" `
  --saida dados/organizados/ofertas_estruturadas.json `
  --texto-bruto dados/raw/texto_bruto.txt
```

O extrator depende do layout do PDF. Inspecione amostras de nomes, IDs de turma, dias e horários antes de converter. O texto extraído está em `dados/raw/texto_bruto.txt` para auditoria.

2.2 JSON intermediário → JSON PAPT

```
python ferramentas/gerador_instancia.py `
  --entrada dados/organizados/ofertas_estruturadas.json `
  --saida dados/convertidos/oferta_convertida.json `
  --id oferta_convertida --carga-min 0 --carga-max 12 `
  --minutos-por-unidade-carga 60 --perfil-preferencias zero
```

A saída deve ficar em `dados/convertidos/` até que sejam confirmados carga individual, qualificação, disponibilidade e preferências. O PDF de oferta sozinho não comprova todos esses valores; os pressupostos adotados ficam em `criterios_conversao`.

2.3 Validar a estrutura

```
python ferramentas/validar_instancia.py modelo/projeto_ic/model_ic/instancias/referencia_v001.json
python ferramentas/validar_instancia.py modelo/projeto_ic/model_ic/instancias/sintetica_media.json
```

O validador verifica o contrato JSON e inconsistências evidentes; não substitui a otimização nem prova toda a viabilidade. O conjunto `turnos_semanais` deve corresponder exatamente aos turnos presentes nos slots.

2.4 Contrato mínimo

Objeto	Campos essenciais
professores	id, nome, carga_min, carga_max, afastado
turmas	id, disciplina, horas, slots
slot	dia, horario, turno; horario_fim opcional
pares_professor_turma	professor, turma, qualificado, disponivel, preferencia

3. Gerar instâncias sintéticas controladas

Exemplo de um cenário médio com 6 professores, 4 disciplinas e 12 turmas:

```
python ferramentas/gerar_instancias.py `
  --saida modelo/projeto_ic/model_ic/instancias --nome sintetica_media `
  --professores 6 --disciplinas 4 --turmas-por-disciplina 3 `
  --densidade-qualificacao 0.60 --disponibilidade 0.80 `
  --perfil-preferencia uniforme --capacidade moderada `
  --conflitos medio --semente 42
```

Fator	O que estudar	Como comparar
Escala	Número de professores, disciplinas e turmas	Aumentar turmas progressivamente e registrar dimensões
Qualificação	Densidade de pares qualificados	Ex.: 0,3; 0,6; 0,9
Disponibilidade	Pares não testemunha disponíveis	Ex.: 0,4; 0,6; 0,8
Preferências	Baixa, moderada, uniforme, alta, polarizada	Mudar somente o perfil e manter a semente
Capacidade	Limites de carga estritos/moderados/folgados	Manter demais características fixas
Conflito	Universo de horários e sobreposições	Controlar slots e registrar conflitos detectados

O gerador cria uma alocação-testemunha; seus pares são qualificados e disponíveis, e os limites de carga são construídos em torno dessa alocação. Isso ajuda a começar com casos viáveis, mas alterações manuais podem invalidar a garantia.

Estratégia de progressão

- A — Referência: 3 professores, 4 turmas. Corrigir erros de formulação antes de seguir.
- B — Pequena: verificar construção e restrições em instância sintética menor.
- C — Média: estudar preferências e capacidade com mesma estrutura e semente.
- D — Crescimento: aumentar turmas/professores em passos e preservar proporções.
- E — Estresse: qualificacão rara, capacidade estrita, conflitos altos e casos inviáveis separados.

Use IDs versionados (por exemplo, `real_2026_2_v001.json`) e evite sobrescrever a referência. A opção `--all` lê somente os JSONs diretamente presentes em `instancias/`; mantenha diagnósticos e dados históricos fora desse diretório.

4. Gestão da bateria de experimentos

Comandos do modelo

```
cd modelo/projeto_ic/model_ic
julia --project=. -e 'using Pkg; Pkg.instantiate()'
julia --project=. PAPT_JuMP.jl referencia_v001.json --time-limit=3600 --gap=0.0 --threads=1
julia --project=. PAPT_JuMP.jl --all --time-limit=600 --gap=0.01 --threads=1
```

Parâmetros:

- `--time-limit=600`: limita cada otimização a 600 segundos.
- `--gap=0.01`: alvo de GAP relativo de 1%.
- `--gap=0.0`: solicita prova de otimalidade dentro do limite, sem garantir que seja concluída.
- `--threads=1`: usa uma thread; `--threads=0` deixa o CPLEX escolher automaticamente.

O que guardar por execução

Grupo	Dados
Identificação	ID da instância, versão/hash JSON, data, repetição e tipo de experimento
Ambiente	Versões Julia/CPLEX/modelo, número de threads, limite de tempo, GAP alvo
Tamanho	Professores, turmas, disciplinas, slots, pares, conflitos
Desempenho	Tempo de carga, construção, exportação LP, tempo de parede, tempo do solver e tempo total
Qualidade	Status, solução incumbente, objetivo, bound, GAP final e componentes F, s_min, C, g_max

Repetições e comparabilidade

Faça uma execução de aquecimento para carregar bibliotecas e absorver compilação JIT. Depois repita cada caso de 3 a 5 vezes na mesma máquina e com as mesmas versões e configurações. Registre todos os resultados; reporte mediana e, se apropriado, média/desvio-padrão. Não compare instâncias de formulações diferentes na mesma análise sem marcar essa diferenca.

O executor exporta um `.lp`, um JSON de métricas por instância e `resultados/resumo_execucoes.csv`. O modelo atual é ponderado: `Z = 0.60 F + 0.20 s_min - 0.10 C - 0.10 g_max`. Algumas referências descrevem a variante lexicográfica, que não é equivalente.

5. Medir tempo usando apenas Julia

A biblioteca padrão de Julia oferece `time_ns()`; medir a diferença e dividir por `1e9` converte nanossegundos em segundos:

```
inicio = time_ns()
# execute o trecho que deseja medir
segundos = (time_ns() - inicio) / 1e9
println("Tempo: ", segundos, " s")
```

Para um trecho curto, `@elapsed` é uma alternativa compacta. No modelo consolidado, os intervalos são separados:

Métrica	O que inclui
tempo_carregamento_s	Leitura e validação do JSON
tempo_construcao_s	Criação das variáveis/restrições e configuração do solver
tempo_exportacao_lp_s	Gravação do modelo linear em arquivo LP
tempo_otimizacao_parede_s	Intervalo de parede em torno de `optimize!(model)`
tempo_otimizacao_solver_s	Tempo retornado pelo solver por `solve_time(model)`
tempo_total_s	Soma das fases acima; não inclui toda a saída e gravação final

```
inicio = time_ns()
optimize!(model)
tempo_parede = (time_ns() - inicio) / 1e9
tempo_solver = solve_time(model)
```

Tempo de parede e tempo informado pelo solver podem divergir. Mantenha ambas as métricas no CSV. O primeiro uso de Julia pode incluir compilação/JIT; por isso, faça aquecimento antes das repetições que serão reportadas.

Evite comparações injustas

• Use o mesmo limite de tempo, número de threads, versões, configuração do solver e máquina.

• Para experimentos que alteram um fator, mantenha fixos os outros dados e a semente.

• Não registre execução interrompida como tempo zero; anote status, limite e se há incumbente.

• Não confunda tempo de leitura/construção com o tempo gasto pelo CPLEX na otimização.

6. Interpretar GAP, bound e status

O executor registra `objective_bound(model)` e `relative_gap(model)` quando estes dados estiverem disponíveis. Em maximização, o bound fornece um limite superior para a melhor solução possível na busca corrente. A incumbente é a melhor solução viável conhecida. O GAP representa a diferença entre a incumbente e o bound segundo a convenção do solver.

Resultado	Como registrar
Ótimo comprovado	Status de otimalidade, objetivo, bound e GAP final (normalmente 0 ou muito próximo)
Tempo esgotado com solução	Manter objetivo incumbente, bound, GAP final e limite de tempo
Sem solução viável	Não tratar objetivo como resultado; investigar restrições/dados e status
GAP indisponível	Registrar vazio/não informado, nunca substituir por zero

Um GAP baixo não diz, sozinho, que duas instâncias têm a mesma dificuldade ou que os modelos são comparáveis. Registre o tamanho e a formulação usada. Para uma comparação experimental, use também o tempo, o status, o bound, o objetivo e os componentes da função objetivo.

Estrutura mínima da tabela experimental

O CSV incluído já contém uma linha de referência por enumeração com dimensões e valores `F=17`, `s_min=3`, `C=8`, `g_max=1`, `Z=9.9`; outra linha é um modelo vazio para duplicar por execução. Campos de tempo, bound e GAP do solver ficam vazios para a linha de enumeração, pois não foram medidos por CPLEX.

Checklist final

☐ JSON validado, identificador/versionamento preservado e fontes/hipóteses documentadas.

☐ Referência executada novamente após cada alteração no modelo.

☐ Tempo de parede, tempo de solver, bound e GAP registrados separadamente.

☐ Configuração de limite de tempo, GAP alvo e threads registrada.

☐ Instâncias inviáveis de propósito separadas dos testes de desempenho.

☐ Resultados da função ponderada separados dos resultados lexicográficos.

Referências técnicas

JuMP API: https://jump.dev/JuMP.jl/stable/api/JuMP/ | Soluções: https://jump.dev/JuMP.jl/stable/manual/solutions/ | CPLEX.jl: https://github.com/jump-dev/CPLEX.jl